

אוניברסיטת תל אביב
סמסטר א' תשפ"ו

מבני נתונים

0368-2158

פרויקט מספר 2

ערימות מתקדמות

איתי בן שחר: 211325287

רוני שירי: 207334525

תוכן עניינים

3	 חלק א' – סקירת מתודות
3	 insert(int key, String info)
3	 findMin()
3	 deleteMin()
4	 decreaseKey(HeapItem x, int diff)
4	 delete(HeapItem x)
4	 meld(Heap heap2)
5	 size()
5	 numTrees()
5	 numMarkedNodes()
5	 totalLinks()
5	 totalCuts()
6	 totalHeapifyCosts()
6	 מתודות עזר:
6	 addRoot()
6	 removeRoot(HeapNode node)
6	 mergeRootLists(HeapNode a, HeapNode b)
6	 linkTrees(HeapNode x, HeapNode y)
7	 successiveLinking()
7	 cut(HeapNode node)
7	 cascadingCut(HeapNode node)
8	 heapifyUp(HeapNode node)
8	 חלק ב' – ניסוי תיאורטי
8	 שאלה 1:
11	 שאלה 2:
12	 שאלה 3:
12	 שאלה 4:
14	 שאלה 5:

חלק א' – סקירת מתודות

insert(int key, String info)

- שימוש: המתודה יוצרת צומת חדש ומכניסה אותו כערימה בדרגה אפס אל רשימת השורשים. כמו כן היא מבצעת פעולת meld בהתאם לתנאי הערימה הקיימת, ולבסוף מעדכנת את המינימום במידת הצורך. לבסוף המתודה מחזירה את הצומת החדש.
- סיבוכיות זמן ריצה: addRoot(node) רצה בסיבוכיות $O(1)$. במידה `lazyMelds = false` יהיה שימוש ב-successiveLinking בסיבוכיות של $O(n)$ במקרה של `lazyDecreaseKey`, או $O(\log(n))$ אחרת, כפי שיפורט בהמשך.
- אלגוריתם: המתודה מתחילה ביצירת אובייקטים מסוג HeapNode ו-HeapItem. לאחר מכן, היא מוסיפה את הצומת אל רשימת השורשים, מעדכנת את המינימום במידת הצורך, ומבצעת successiveLinking במידה ושדה ה-meld של הערימה מתאים ל-eager meld.

findMin()

- שימוש: מחזירה את האיבר בעל המפתח הקטן ביותר.
- סיבוכיות זמן ריצה: $O(1)$ שדה המתוחזק באובייקט הערימה.
- אלגוריתם: החזרה של שדה שמור.

deleteMin()

- שימוש: מחיקת האיבר המינימלי בערימה.
- סיבוכיות זמן ריצה: כעלות פעולת successiveLinking. כלומר, $O(\log(n))$ במקרה שבו `lazyDecreaseKey = false`, אחרת $O(n)$.
- אלגוריתם:
 - המתודה מתחילה בבדיקת מקרה קצה – האם הערימה ריקה.
 - אם לצומת המינימלי יש ילדים, היא עוברת על הילדים, מנתקת אותם מן השורש, ומעדכנת את שדה marked שלהם. ה-rank חסום על ידי $O(\log(n))$, ולכן מדובר בעלות $O(\log(n))$.
 - לאחר מכן, היא מאחדת את רשימות השורשים המקורית וזו של הילדים על ידי קריאה ל-mergeRootLists ב- $O(1)$.

- כמו כן, היא קוראת למתודות העזר removeRoot ב- $O(1)$, שמעדכנת את המצביעים ברשימת השורשים כך שהשורש שנמחק לא יהיה חלק ממנה.
- לבסוף, היא מבצעת קריאה ל-successiveLinking.

:decreaseKey(HeapItem x, int diff)

- שימוש: בהינתן צומת, והפרש רצוי, המתודה מקטינה את ערכו של המפתח בהפרש הנתון, תוך שמירה על מבנה חוקי של הערימה.
- סיבוכיות זמן ריצה: במקרה של eager decreaseKey יהיה כעלות heapifyUp, כלומר $O(\log(n))$, אחרת $O(n)$ כעלות cascadingCut.
- אלגוריתם: המתודה מבצעת הקטנת ערך המפתח. במידה והמפתח החדש קטן מזה של אביו, מבוצע תיקון בהתאם לאופי העץ ע"י קריאה למתודה המתאימה מבין cascadingCut ו-heapifyUp. לבסוף היא מעדכנת את המינימום.

:delete(HeapItem x)

- שימוש: בהינתן צומת, והפרש רצוי, המתודה מוחקת אותו מן הערימה, תוך שמירה על המבנה חוקי של שלה.
- סיבוכיות זמן ריצה: כתלות ב-decreaseKey, במידה והוא eager, שתי המתודות deleteMin ו-decreaseKey ירוצו ב- $O(\log(n))$, אחרת שתיהן ירוצו ב- $O(n)$ ב-W.C.
- אלגוריתם: המתודה מבצעת פעולת decreaseKey לערך שלילי, כך שצומת זה מחזיק כעת במפתח המינימלי. לאחר מכן מתבצעת קריאה למתודה deleteMin.

:meld(Heap heap2)

- שימוש: בהינתן ערימה אחרת, המתודה מבצעת פעולת meld לפי אופי הערימה הנתונה.
- סיבוכיות זמן ריצה: כתלות בשדה lazyMelds של הערימה. $O(1)$ במקרה של lazy, אחרת כעלות פעולת successiveLinking. כלומר, $O(n)$ ב-W.C. במקרה של lazyDecreaseKey ו- $O(\log(n))$ אחרת.
- אלגוריתם:
 - המתודה מתחילה בעדכון השדות – במידה והערימה ריקה – השדות החדשים יהיו אלה של הערימה בארגומנט. אחרת, תתבצע קריאה ל-mergeRootLists ב- $O(1)$ והשדות ייסכמו.

- לאחר מכן מתבצע עדכון המינימום של הערימה.
- לבסוף, במידה ואופי ה-meld של הערימות אינו עצל, מתבצעת קריאה ל-successiveLinking.

:size()

- שימוש: מחזירה את גודל (מספר הצמתים) בערימה.
- סיבוכיות זמן ריצה: $O(1)$ שדה המתוחזק באובייקט הערימה.
- אלגוריתם: החזרה של שדה שמור.

:numTrees()

- שימוש: מחזירה את האיבר מספר העצים בערימה, כלומר, מספר השורשים.
- סיבוכיות זמן ריצה: $O(1)$ שדה המתוחזק באובייקט הערימה.
- אלגוריתם: החזרה של שדה שמור.

:numMarkedNodes()

- שימוש: מחזירה את מספר הצמתים המסומנים כתוצאה מפעולת cascadingCut.
- סיבוכיות זמן ריצה: $O(1)$ שדה המתוחזק באובייקט הערימה.
- אלגוריתם: החזרה של שדה שמור.

:totalLinks()

- שימוש: מחזירה את סך כל החיבורים שקרו בהיסטוריה של הערימה הנוכחית.
- סיבוכיות זמן ריצה: $O(1)$ שדה המתוחזק באובייקט הערימה.
- אלגוריתם: החזרה של שדה שמור.

:totalCuts()

- שימוש: מחזירה את מספר הניתוקים הכולל בין צמתים להוריהם בהיסטוריה של הערימה הנוכחית.
- סיבוכיות זמן ריצה: $O(1)$ שדה המתוחזק באובייקט הערימה.
- אלגוריתם: החזרה של שדה שמור.

`:totalHeapifyCosts()`

- שימוש: מחזירה את סך כל פעולות ה-`heapifyUp` כתוצאה מפעולת `decreaseKey`.
- סיבוכיות זמן ריצה: $O(1)$ שדה המתוחזק באובייקט הערימה.
- אלגוריתם: החזרה של שדה שמור.

מתודות עזר:

`:addRoot()`

- שימוש: בהינתן צומת, המתודה מוסיפה אותו אל רשימת השורשים.
- סיבוכיות זמן ריצה: כל הפעולות מתבצעות ב- $O(1)$, `mergeRootLists` מתבצעת גם בכן ב- $O(1)$.
- אלגוריתם: המתודה מבצעת בדיקת מקרי קצה, ואם כרגע לא קיים שורש בערימה, מעדכנת את הצומת להיות השורש. אחרת, מבצעת קריאת למתודת `mergeRootLists`, ולאחר מכן מעדכנת את המינימום במידת הצורך.

`:removeRoot(HeapNode node)`

- שימוש: בהינתן צומת ברשימת השורשים, המתודה מסירה אותו.
- סיבוכיות זמן ריצה: עדכון מצביעים ב- $O(1)$.
- אלגוריתם: המתודה מבצעת עדכון מצביעים ועדכון שדות. כלומר, הצמתים `next` ו-`prev` של השורש שנמחק יצביעו אחד אל השני.

`:mergeRootLists(HeapNode a, HeapNode b)`

- שימוש: בהינתן שתי רשימות של שורשים, המתודה מאחדת אותם לרשימת שורשים אחת.
- סיבוכיות זמן ריצה: עדכון מצביעים ב- $O(1)$.
- אלגוריתם: המתודה מבצעת עדכון מצביעים ועדכון שדות. כלומר, נשברת המעגליות של כל אחת מרשימות השורשים, ומתבצע חיבור 'זנב' של רשימה אחת ל-'ראש' של רשימה אחרת.

`:linkTrees(HeapNode x, HeapNode y)`

- שימוש: בהינתן צומת x ו-צומת y כך ש- $y < x$ ודרגתם r , המתודה מחברת אותם לעץ יחיד בדרגה $r + 1$.

- סיבוכיות זמן ריצה: עדכון מצביעים ושדות המתבצעים ב- $O(1)$.
- אלגוריתם: מתבצע עדכון השדות והמצביעים, וחיבור של y אל רשימת הילדים של x .

successiveLinking()

- שימוש: המתודה מבצעת איחוד של שורשים מ-rank זהה, כך שבסוף התהליך לא יהיו שורשים מאותה הדרגה.
- סיבוכיות זמן ריצה: מתבצעת סריקה על רשימת השורשים ב- $O(n)$ ב-W. C. ו- $O(\log(n))$ אמורטייזד. כמו כן, במידה ושדה lazyDecreaseKey הוא false, הרי שמספר השורשים חסום על ידי $O(\log(n))$.
- אלגוריתם: המתודה מאתחלת מערך של צמתים בגודל $O(\log(n))$ ומתחילה לרוץ על רשימת השורשים שאורכה $O(n)$.
 - מתבצע ניתוק של כל צומת שהיא עוברת עליו.
 - אם לא קיים במערך צומת בעל הדרגה הזו – הוא יאוכסן בה. אחרת, יתבצע חיבור על ידי קריאה למתודת linkTrees.
 - המתודה תמשיך לרוץ עד שתשלים סיבוב שלם.

cut(HeapNode node)

- שימוש: בהינתן צומת, המתודה מנתקת אותו מאביו ומוסיפה אותו אל רשימת השורשים.
- סיבוכיות זמן ריצה: עדכון מצביעים ושדות ב- $O(1)$, במידה ו-lazyMelds = false כעלות פעולת successiveLinking, כלומר $O(n)$.
- אלגוריתם: המתודה מנתקת את ההורה מאביו על ידי עדכון השדה parent, מסירה את הצומת הנתון מרשימת הילדים של ההורה, ומעדכנת את דרגתו. לאחר מכן, הצומת מצביע על עצמו ומוכנס לרשימת השורשים על ידי קריאה ל-addRoot, שרצה כאמור ב- $O(1)$. לבסוף, אם שדה lazyMelds = false, מתבצעת קריאה ל-successiveLinking.

cascadingCut(HeapNode node)

- שימוש: בהינתן צומת שנמחק ילדו, המתודה הופכת אותו למסומן במידה והוא לא היה לפני כן. אחרת היא מנתקת אותו מאביו, וקוראת לעצמה שוב באופן רקורסיבי על ה-parent שלו.
- סיבוכיות זמן ריצה: כעומק העץ, שבמקרה זה הוא $O(n)$.

- אלגוריתם: המתודה רצה באופן רקורסיבי, כאשר תנאי העצירה שלה הוא בדיקה של מצב ה-`marked` של הצומת שקיבלה כארגומנט. במידה והוא לא היה מסומן קודם – יש לסמנו ולסיים. אחרת, יש לנתק אותו גם כן מן העץ, ולהוסיפו לרשימת השורשים על ידי קריאה למתודת `cut`, וביצוע קריאה רקורסיבית על אביו.

`heapifyUp(HeapNode node)`:

- שימוש: בהינתן צומת, במידה והמפתח שלו קטן מזה של ההורה שלו מתבצעת ביניהם החלפה. בעת הגעה לשורש או לצומת שאביו בעל מפתח קטן, המתודה מפסיקה.
- סיבוכיות זמן ריצה: המתודה רצה על גובה העץ, שבמקרה זה הוא $O(\log(n))$.
- אלגוריתם: המתודה מבצעת השוואה בין המפתח של הצומת הנוכחי לזה של ההורה, וכן בודקת שאיננו השורש. במידה והמפתח של ההורה גדול מזה של המפתח הנוכחי, מתבצע עדכון מצביעים של ה-`HeapItem`.

חלק ב' – ניסוי תיאורטי

שאלה 1:

נשתמש בהנחה שלכל אורך סדרת הפעולות מובטח שמספר האיברים בערימה בכל רגע הוא לכל היותר n . משום שב- "ערימה בינומית עם ניתוקים":

1. `lazyMelds = false` כלומר, בפעולות שמבצעים בהן `meld` – הכנסה לדוגמא, מבצעים `successiveLinking`, וחוזרים למבנה של ערימה בינומית רגילה.
2. `lazyDecreaseKeys = true` כלומר, `decreaseKey` מתבצע כמו בערימת פיבונאצ' – עם סימון צמתים וביצוע `Cascading Cuts`.

כפי שצוין בהנחיות, `successiveLinking` היא פעולה שתמיד רצה בזמן $O(\log(n))$, כי בכל מקרה בסוף צריך לעבור על מערך ה"סלים" שאורכו לוגריתמי, לקחת מכל תא לא ריק את העץ ששמור בו, ולצרף את העצים לרשימת שורשים אחת.

בנוסף, ב-`successiveLinking` ובכל `cut` מספר השורשים לאחר ביצוע `consolidate` הוא $O(\log(n))$ מכיוון שאין יותר משורש אחד מכל דרגה. לכן גם מספר החיתוכים ב-`cascadingCuts` חסום ע"י אותו החסם.

תחילה, נגדיר פונקציית פוטנציאל בדומה לניתוח פיבונאצ' :

$$\Phi(H) = t + 2 \cdot m$$

כאשר:

- t מספר העצים ברשימת השורשים
- m מספר הצמתים המסומנים בערימה

ניתוח הפעולות:

1. insert(key, info)

- מצב לפני: ערימה עם t שורשים, ו- m איברים מסומנים.
 - עלות בפועל: מוסיפים עץ יחיד, מבצעים meld ואז successiveLinking בעלות $O(\log(n))$.
 - שינוי פוטנציאל: מספרים הצמתים המסומנים לא משתנה בהכנסה.
- הבחנה: כל הפעולות יבצעו בסיומן successiveLinking, לכן, מספר העצים לפני ההכנסה הוא בהכרח $O(\log(n))$. בגלל שמתבצעת כאן פעולת successiveLinking, הרי שבסיום ההכנסה מספר העצים יישאר $O(\log(n))$, ולכן השינוי במספר העצים הוא $O(\log(n))$.
- אם כן, סה"כ:

$$amort(insert) = O(\log n) + O(\log(n)) = O(\log(n))$$

2. findMin()

- מכיוון שהמחלקה מתחזקת מצביע למינימום של הערימה, העלות בפועל היא $O(1)$, אין לפעולה שום השפעה על הפוטנציאל לכן השינוי בפוטנציאל הוא 0.
- סה"כ:

$$amort(findMin) = O(1)$$

3. deleteMin()

- מצב לפני: נניח כי צומת min הוא שורש עם דרגה d . מתקיים: $d = O(\log(n))$ משום שמבנה הערימה הבינומית נשמר לאחר כל פעולה.
 - עלות בפועל:
 - i. מוסיפים את d ילדיו של min לרשימת השורשים, כלומר $O(d)$ חיבור רשימות.
 - ii. מריצים successiveLinking על כל השורשים ב- $O(\log(n))$.
- סה"כ:

$$O(\log(n) + d) = O(\log(n))$$

• שינוי פוטנציאל:

- לפני הפעולה היו $O(\log(n))$ עצים בערימה, כפי שהוסבר בניתוח פעולת ההכנסה. לפני ביצוע successiveLinking, כמות העצים גדלה ב- d . שוב, עקב ביצוע הפעולה הזו מספר העצים לאחר מכן הוא $O(\log(n))$, סה"כ $O(\log(n))$ שינוי במספר העצים.
- נסמן את מספר הילדים המסומנים של המינימום ב- k . כולם הפכו ללא מסומנים לאחר הוספתם לרשימת השורשים, ולכן השינוי בפוטנציאל הצמתיים המסומנים הוא $-k$.

סה"כ:

$$amort(deleteMin) = O(\log(n)) + O(\log(n)) = O(\log(n))$$

4. decreaseKey(x, diff)

שני תת-מקרים:

1. אם הקטנת המפתח לא הובילה לחיתוך (המפתח עדיין מקיים את תנאי הערימה)
 - עלות בפועל $O(1)$.
 - אין שינוי בפוטנציאל.
 - עלות אמורטייזד $O(1)$.
 2. אם הקטנת המפתח הובילה להפרה של תנאי הערימה, חותכים את הצומת ומבצעים קריאה ל-cascadingCut. כאן כל חיתוך גורר אחריו meld ולכן successive linking. נסמן ב- k את מספר החיתוכים לכל פעולה. שוב מאותה סיבה של שמירה על מבנה הערימה לאחר כל פעולה, ידוע ש- $k = O(\log(n))$
 - עלות ממשית לחיתוך יחיד $O(1)$, ועוד $O(\log(n))$ ל-successiveLinking.
 - לכן, עלות ממשית ל- k חיתוכים $O(k \cdot \log(n))$ שלפי החסם על $k - O(\log^2(n))$.
 - שינוי פוטנציאל בחיתוך יחיד:
- הצומת הנחתך: אם צומת זה היה מסומן הפוטנציאל יורד ב-2. אחרת, אין שינוי במספר הצמתיים המסומנים עבור חיתוך יחיד.
 - ההורה ממנו נחתך: אם ההורה לא היה מסומן הוא הופך למסומן והפוטנציאל עולה ב-2, אם היה מסומן הוא יחתך בפעולה הבאה.

○ שינוי במספר העצים $O(\log(n))$ משום שלפני ואחרי הפעולה של החיתוך, כמות העצים היא $O(\log(n))$.

- בסה"כ בביצוע k חיתוכים, אם הצומת שנחתך היה מסומן אז $k - 1$ הצמתים האחרים היו מסומנים, וכעת אינם מסומנים – ירידה בפוטנציאל של $2(k - 1)$. אם הצומת לא היה מסומן אז ירידה של $2(k - 2)$. לבסוף, הצומת שבו נעצרו החיתוכים בוודאות לא היה מסומן, לכן מדובר בעליה בפוטנציאל של 2. ובכל מקרה ירידה בפוטנציאל של $O(k)$, ששווה כאמור ל- $O(\log(n))$.

סה"כ נקבל:

$$amort(decreaseKey) = O(\log^2(n)) - O(\log(n)) = O((\log^2(n)))$$

5. delete(x):

- ממומש כ- $decreaseKey(x, x.key + 1)$ ולאחר מכן קריאה ל- $deleteMin$.
- לכן, אם x שורש או שלא מופר תנאי הערימה ולכן אין חיתוכים הפחתת המפתח מתבצעת ב- $O(1)$.
- אם ישנם חיתוכים מהניתוח של $decreaseKey$, נסיק שהעלות אמורטיזד היא $O(\log^2(n))$, ולאחר מכן $deleteMin$ מבצע עוד $O(\log(n))$.

סה"כ: $O(\log^2(n))$ אמורטיזד.

שאלה 2:

טבלה 1 – זמני ריצה אמורטיזד של הפעולות השונות עבור ארבעת סוגי הערימה.

פעולה	ערימה בינומית	ערימה בינומית עזלה	ערימת פיבונאצ'י	ערימה בינומית עם ניתוקים
insert	$O(1)$	$O(1)$	$O(1)$	$O(\log(n))$
findMin	$O(1)$	$O(1)$	$O(1)$	$O(1)$
deleteMin	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$
decreaseKey	$O(\log(n))$	$O(\log(n))$	$O(1)$	$O(\log^2(n))$
delete	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log^2(n))$

שאלה 3:

טבלה 2 – זמן ריצה אסימפטוטי לשלושת ניסויים, עבור ארבעת סוגי הערימות.

ערימה בינומית עם ניתוקים	ערימת פיבונאצ'י	ערימה בינומית עצלה	ערימה בינומית	
$O(n)$	$O(n)$	$O(n)$	$O(n \cdot \log(n))$	ניסוי 1
$O(n \cdot \log(n))$	$O(n \cdot \log(n))$	$O(n \cdot \log(n))$	$O(n \cdot \log(n))$	ניסוי 2
$O(n)$	$O(n)$	$O(n \cdot \log(n))$	$O(n \cdot \log(n))$	ניסוי 3

שאלה 4:

ניסוי 1:

טבלה 3 – מחיקת המינימום לאחר הכנסה בסדר אקראי של מספרים בין 1 ל-464,646 (ממוצע על פני 20 הרצות) עבור ארבעת סוגי הערימות.

ערימה בינומית עם ניתוקים $lm = F; ldk = T$	ערימת פיבונאצ'י $lm = T; ldk = T$	ערימה בינומית עצלה $lm = T; ldk = F$	ערימה בינומית $lm = F; ldk = F$	
53.10	13.65	11.05	61.50	זמן ריצה (msec)
464,653	464,636	464,636	464,653	מספר חיבורים
0	0	0	0	מספר חיתוכים
0	0	0	0	עלות Heapify Up כוללת
9	12	9	9	מספר העצים בסיום
464,645	464,645	464,645	464,645	גודל הערימה בסיום
18	464,636	464,636	18	עלות מקסימלית לפעולה
1.20	1.20	1.34	1.34	עלות ממוצעת לפעולה

ראשית, בדומה לתיאוריה, זמן הריצה של יצירת הערימה ומחיקת המינימום היה נמוך ביותר בערימת פיבונאצ'י. סוגי הערימות עבור פעולת ה-meld שאינה עצלה רצו בזמנים גבוהים משמעותית. בנוסף, כפי שניתן לראות, מכיוון שבניסוי זה מוחקים את המינימום בלבד, הרי שלא מבוצעות כלל פעולות ק heapify up או פעולות חיתוך. מספר החיבורים הכולל דומה עבור ארבעת סוגי הערימות, כאשר עבור ערימות פיבונאצ'י ובינומית עצלה התקבלו פחות חיבורים, שכן פעולת ה-successive linking נעשית רק בסוף, לעומת שתי הערימות האחרות בהן מבוצע

חיבור בכל פעם שיש שני תתי עצים מאותה הדרגה. מכיוון שלא בוצעו פעולות decreaseKey, ההבדל היחיד בין המודלים הוא באופן ה-meld שלהם, ולכן ערימות עם meld זהה קיבלו תוצאות זהות.

ניסוי 2:

טבלה 4 – מחיקת המינימום לאחר הכנסה בסדר אקראי של מספרים בין 1 ל-464,646, ולאחר מכן מחיקת המקסימום עד לערימה בגודל 46 (ממוצע על פני 20 הרצות) עבור ארבעת סוגי הערימות.

ערימה בינומית עם ניתוקים lm = F; ldk = T	ערימת פיבונאצ'י lm = T; ldk = T	ערימה בינומית עצלה lm = T; ldk = F	ערימה בינומית lm = F; ldk = F	
235.30	116.80	398.85	483.45	זמן ריצה (msec)
1,404,524	748,748	7,552,307	7,553,024	מספר חיבורים
748,725	748,706	0	0	מספר חיתוכים
0	0	4,615,788	4,613,177	עלות Heapify Up כוללת
4	4	4	4	מספר העצים בסיום
46	46	46	46	גודל הערימה בסיום
41	464,636	464,636	37	עלות מקסימלית לפעולה
2.32	1.61	13.09	13.09	עלות ממוצעת לפעולה

הפעם, בוצעו כ-46e5 מחיקות, שמומשו באמצעות מתודות DecreaseKey ו-DeleteMin. כתוצאה מכך ההבדלים בזמני הריצה בין ערימת פיבונאצ'י לבינומית היו משמעותיים אף יותר. הפעם, הערימות הבינומיות מבצעות את פעפוע המינימום למעלה באמצעות heapifyUp, ואילו שתי המתודות שמבצעות את הקטנת המפתח באופן עצל מבצעות חיתוכים במקום, כפי במצופה מהתיאוריה.

ניסוי 3:

טבלה 5 – מחיקת המינימום לאחר הכנסה בסדר אקראי של מספרים בין 1 ל-464,646. לאחר מכן הקטנת המפתח לעשירית העליונה של המפתחות ומחיקת המינימום פעם נוספת (ממוצע על פני 20 הרצות) עבור ארבעת סוגי הערימות.

ערימה בינומית עם ניתוקים lm = F; ldk = T	ערימת פיבונאצ'י lm = T; ldk = T	ערימה בינומית עצלה lm = T; ldk = F	ערימה בינומית lm = F; ldk = F	
66.35	14.50	15.35	61.25	זמן ריצה (msec)
511,648	511,571	464,652	464,672	מספר חיבורים

46,998	46,938	0	0	מספר חיתוכים
0	0	158,105	158,044	עלות Heapify Up כוללת
14	11	8	8	מספר העצים בסיום
464,644	464,644	464,644	464,644	גודל הערימה בסיום
18	464,636	464,636	18	עלות מקסימלית לפעולה
1.09	1.09	1.22	1.22	עלות ממוצעת לפעולה

נשים לב כי מספר החיתוכים במקרה של הקטנת מפתח עצלה, שווה בקירוב למספר הקטנות המפתח שבוצעו שכן כל עץ שבוצעה לו הפחתת מפתח נותק מן העץ, וייתכן שאביו נחתך גם הוא.

שאלה 5:

סעיף א:

ישנם הבדלים משמעותיים הנובעים מאופי המימוש (עצל מול רגיל).

ניסוי 1:

ניתן לראות שערימת פיבונאצ'י וערמה בינומית עצלה מהירות יותר לעומת ערימה בינומית רגילה, או בינומית עם ניתוקים. דבר זה נובע מהסיבה ש-lazyDecreaseKey גורר פעולת ההכנסה פשוטה – הוספה לרשימה (זמן קבוע), בעוד שבערימות שמבצעות הכנסה בצורה לא עצלה ושומרות על מבנה הנתונים, מתבצע מיזוג בכל הכנסה שלוקח זמן רב יותר.

ניסוי 2:

ערימת פיבונאצ'י משמעותית מהירה יותר עקב המימוש של המחיקה. משום שהמחיקה קוראת ל-decreaseKey ואז ל-deleteMin, המימוש העצל שלה מאפשר לזמן הכולל של המחיקות להיות מהיר בהרבה, אף על פי שבמחיקת המינימום אנחנו מסדרים את הערימה.

ניסוי 3:

בניסוי זה ההבדל המשמעותי מגיע גם מ-decreaseKey. שכן, בערימות שמבצעות את פעולה זו בצורה לא עצלה, אף על פי שמחיקת המינימום הראשונה מסדרת את הערימה למבנה ה"תקין" שלה, כל פעולת הקטנת מפתח גוררת heapifyUp, שעולה יותר מביצוע חיתוכים מבלי לטפל במבנה הערימה.

סעיף ב:

- בניסוי הראשון: חזינו שכולן יהיו לינאריות, ואכן הזמנים באותו סדר גודל (מאות מילי-שניות), ללא התפוצצות אקספוננציאלית.

- בניסוי השני: חזינו התנהגות אסימפטוטית זהה עבור כל סוגי הערימות, אך מכיוון שמדובר בהתנהגות אסימפטוטית, ולא אמורטייזד, הרי שהעלות בפועל לסדרה הנתונה נמוכה יותר עבור ערימת פיבונאצ'י, שכן המימוש שלה יעיל יותר, ולכן הקבוע האסימפטוטי ב-W.C. נמוך יותר גם הוא.
- בניסוי השלישי: כפי שניתן לראות מטבלה 5, לערימה בינומית רגילה יש כ-114e3 פעולות של heapifyUp, לעומת 46e3 ניתוקים בערימת פיבונאצ'י, ואכן הזמן שהתקבל מהיא יותר פי 2.

סעיף ג:

ישנם הבדלים משמעותיים בין העלות המקסימלית לזו הממוצעת בין ערימות שבהן מתבצעת הקטנת מפתח בצורה עצלה ולערימות בהן הפעולה מתבצעת בצורה שאינה עצלה.

- בניסוי הראשון: הדבר מתבטא בכך שהערימות העצלות (מבחינת מיזוג) מכניסות את האיברים בזמן מהיר, ופעולת מחיקת המינימום מסדרת את כל הערימה בפעולה אחת. כלומר, פעולת deleteMin יחידה משלמת את המחיר כולו ולכן העלות המקסימלית מאוד גבוהה, אך בממוצע העלות נמוכה. מנגד, הערימות הלא-עצלות (מבחינת מיזוג) שומרות על המבנה בכל הכנסה, ולכן העבודה "נפרשת" על כלל הפעולות וההבדל בין העלות המקסימלית למינימלית לא משמעותי.
 - בניסוי השני: העלות המקסימלית עבור ה-meld העצל היא עדיין זו של מחיקת המינימום לראשונה, לאחר מכן כל מחיקת מינימום מבצעת decreaseKey וחיתוכים עוקבים, אך אף חיתוך אינו גורם לניתוק של 464,646 צמתים. לכן, שוב ההבדל בעלות ההמוצעת לפעולה יהיה משמעותי ביותר עבור פעולות אלו.
 - בניסוי השלישי: באופן דומה ההבדל מתבטא בכך שהערימות העצלות (מבחינת הקטנת מפתח) מבצעות את הקטנת המפתח בזמן מהיר יחסית (ומבצעות רק חיתוך או מספר חיתוכים). פעולת מחיקת המינימום מסדרת את כל הערימה בפעולה אחת יקרה, בעוד שבערימות הלא-עצלות (מבחינת הקטנת מפתח) הערימה שומרת על המבנה עם כל הקטנת מפתח, ולכן העבודה "נפרשת" על כלל הפעולות וההבדל בין העלות המינימלית לממוצעת נמוך יותר.
- לבסוף, ניתן לשים לב כי העלות הממוצעת לפעולה של ערימת פיבונאצ'י עבור כל אחד מהניסויים קטנה מהעלות הממוצעת של כל שאר הערימות, ולכן יעילה יותר כצפוי, בייחוד במקרה של מחיקות (ניסוי 2).